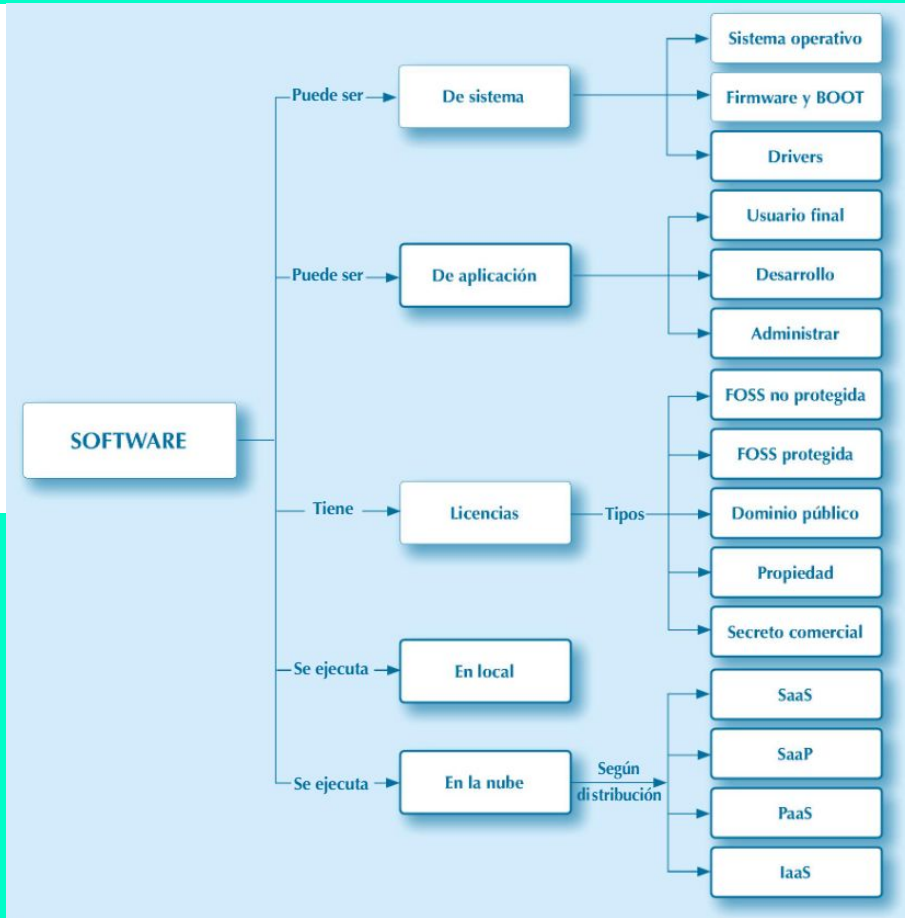
UD3 – Software relacionado con sistemas operativos



3.1. Definición y clasificación del software

¿Qué es?

Conjunto de instrucciones que indican al hardware qué hacer. El SO (sistema operativo) gobierna los recursos y los cede temporalmente a las aplicaciones.

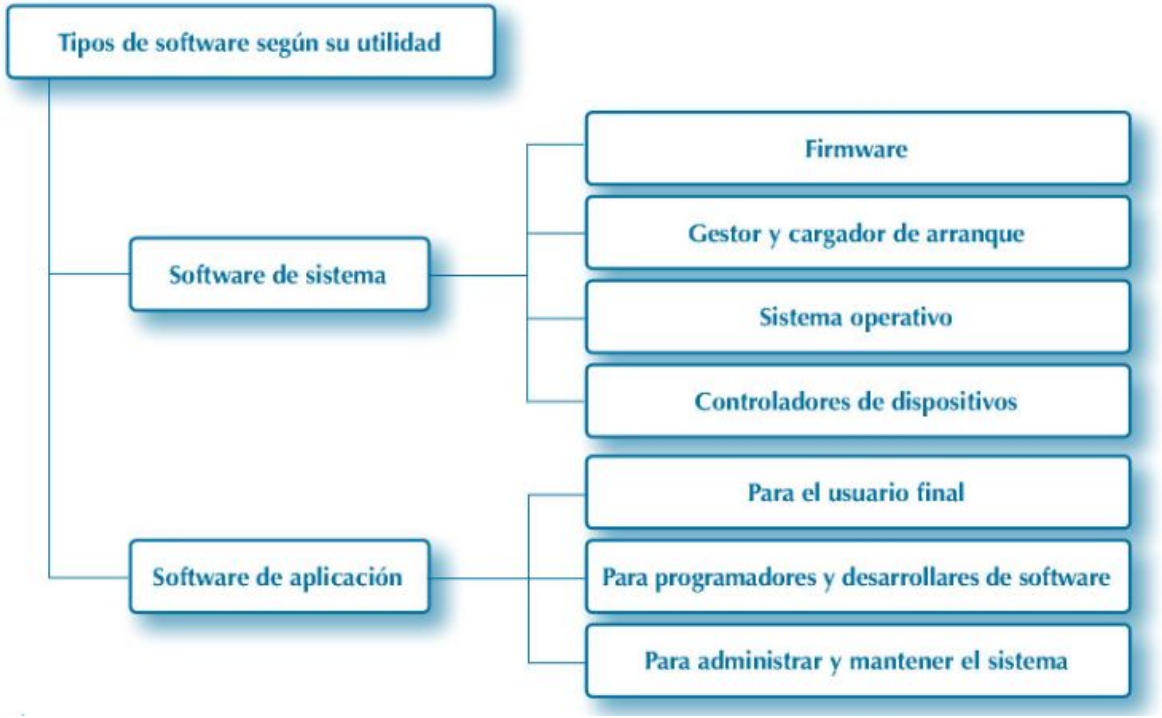
Según su utilidad

- Software de sistema: imprescindible para que el SI funcione (SO, drivers, utilidades básicas).
- Software de aplicación: programas que usa el usuario para realizar tareas (ofimática, navegador, edición…).

Según dónde se ejecuta

- Local: corre en el equipo del usuario (consume CPU/RAM/almacenamiento locales).
- Remoto / nube: corre en servidores externos; el usuario accede vía red (SaaS, apps web, VDI).

3.1.1. Tipos de software según su utilidad



Se distinguen dos tipos de software según utilidad:

Software de Sistema:

Imprescindible para que el sistema funcione.

Software de aplicación:

El que el usuario emplea para realizar sus tareas.

3.1.1.A. SOFTWARE DEL SISTEMA

¿Qué es?

Imprescindible para que el hardware arranque y funcione correctamente.

¿Qué incluye?

- Firmware/UEFI y su interfaz de configuración.
- Gestor/cargador de arranque (Boot Manager/GRUB).
- Sistema operativo (kernel y servicios básicos).
- Controladores de dispositivos y utilidades de administración.

Puntualizaciones

Es imprescindible y, por lo general, se entrega **ya configurado**.

El administrador **personaliza y ajusta**; suele ser **transparente** para el usuario final.

Debe **mantenerse actualizado** (firmware, kernel, drivers) para garantizar estabilidad y seguridad.

3.1.1.B. SOFTWARE DE APLICACIÓN

¿Qué es? El es software necesario para que usuarios, desarrolladores y administradores realicen sus tareas. Usaremos **“aplicación”** como abreviatura de **software de aplicación** (uso extendido y correcto), normalmente accesible mediante **GUI**.

Tipos (según el rol)

1. **Usuario final** → Ofimática, edición gráfica/foto, multimedia, entretenimiento, comunicación, ERP, e-learning.(Ej.: LibreOffice, GIMP, VLC, Firefox, Moodle App)



2. **Programadores/desarrolladores** → Editar código, compilar, probar; frecuentemente en un **IDE**. (Ej.: VS Code/VSCodium, IntelliJ, Eclipse, Git, Docker).



3. **Administración y mantenimiento** → Recuperación ante fallos, actualizaciones, gestión de usuarios/archivos/permisos, hardware/redes, pruebas de seguridad. (Ej.: mmc/PowerShell, GNOME Disks, Ansible, Wireshark, nmap).



3.1.2. Tipos de SW según el lugar de los recursos utilizados

Clasificamos el software por dónde se procesa su carga de trabajo:

Software local: se instala y ejecuta en el equipo del usuario (PC, móvil, IoT).

- Rendimiento inmediato y uso de CPU/RAM/almacenamiento locales.
- Requiere instalación/mantenimiento; depende del SO y del hardware.

Software en la nube: se ejecuta en servidores remotos y se accede por red.

- Cero instalación local (SaaS), colaboración, acceso ubicuo.
- Depende de Internet; considerar privacidad/ubicación de datos.

👆 Regla práctica: ¿Dónde computa la mayor parte del trabajo? Ahí lo clasificas (local vs nube).

3.1.2.A. Software Local

¿Qué es? Aplicación instalada en el sistema del usuario y dependiente del SO (Windows, Linux, Android, iOS…).

Características

- Accede (con permisos del SO) a recursos locales: cámara, micrófono, GPS, disco, red, GPU…
- Móvil/IoT: “apps” locales que a menudo consumen servicios web (mixto cliente-local + backend-nube).
- Interoperabilidad: posibles técnicas para ejecutar apps de otro SO (capas de compatibilidad, VM, contenedores), pero siempre hay dependencia del entorno.
- Mantenimiento y seguridad: requiere actualizaciones periódicas (app y SO), gestión de permisos y origen confiable del paquete (repo/tienda).
- Dependencia de hardware local.

3.1.2.B. Software en la nube

¿Qué es? Procesamiento y datos en servidores remotos (acceso por Internet); el usuario usa navegador o app ligera.

Características

- Cómputo/almacenamiento fuera del equipo.
- Acceso ubicuo y colaboración.
- Actualizaciones y copias a cargo del proveedor.
- Dependencia de conectividad y políticas de datos.

Ejemplos (genéricos)

Ofimática online, CRM web, edición multimedia en navegador, chat/videollamada.

Modelos: SaaS, SaaP, PaaS, IaaS (Ver en diapositivas siguientes).

3.1.3. Clasficación según dos parámetros

Usaremos dos ejes distintos para clasificar 4 conceptos:

1) **Eje “dónde se ejecuta”**

   - Nube: el cómputo principal está en servidores remotos.
   - Local: el cómputo principal está en el equipo del usuario.

2) **Eje “cómo se entrega/licencia”**

   - Servicio (suscripción, gestionado por proveedor).
   - Producto (instalación local, gestionado por el usuario).

3.1.3.A. SaaS (Software as a Service)

Aspecto	Resumen
Qué es	Aplicación cuyo **cómputo y datos** residen en **servidores remotos**; acceso por **web** o app cliente vía **Internet/red**.
Licencia y operación	Uso por **suscripción** (mensual/anual) o licencia; **mantenimiento/actualizaciones** a cargo del proveedor.
Ejemplo	**Correo web** (p. ej., **Gmail**, **Outlook.com**, **Proton Mail**, **Zoho Mail**): el servidor gestiona buzones y envío; el usuario accede con **navegador**.
Ventajas	Sin instalación local; **acceso ubicuo** y colaboración; copias y parches gestionados por el proveedor.
Riesgos / consideraciones	**Dependencia del proveedor** (portabilidad/migración); **privacidad y seguridad** de datos alojados en el servicio (accesos/ataques).

3.1.3.B. SaaP (Software as a Product***)

Aspecto	Resumen breve
Qué es	**SaaP = Software como Producto**: se **instala y ejecuta en local**; datos **en tu equipo** (salvo sincronización opcional).
Licencia/operación	**Licencia por versión/perpetua**; instalación por el usuario (descarga/tienda); **actualizaciones** via SO o manuales.
Ejemplo	LibreOffice, GIMP, VLC, OnlyOffice Desktop.
Ventajas	**Control y privacidad**, **modo offline**, rendimiento según **CPU/GPU/RAM/SSD** del equipo.
Riesgos	Tú te ocupas de **copias, parches y seguridad**; **dependencia de drivers/hardware**; posibles **formatos propietarios**.

3.1.3.C. PaaS (Platform as a Service)

Aspecto	Resumen
Qué es	**PaaS (Plataforma como Servicio)**: plataforma **gestionada** donde desplegar **tu código** (o contenedores) sin administrar SO/infra. El proveedor aporta **runtime**, librerías, BD, logs, red y **escalado**.
Licencia y operación	Normalmente **suscripción/por uso** en la nube. Subes el **código/imagen** y la plataforma lo ejecuta/escala. Puede existir variante **on-prem** con soluciones de plataforma.
Ejemplo	**Heroku**, **Google App Engine**, **Azure App Service**, **Red Hat OpenShift** (como plataforma de contenedores).
Ventajas	Menos **operaciones** y mantenimiento; **despliegue rápido**, **escalado automático**, add-ons (BD/caché/colas), **portabilidad** al empaquetar dependencias.
Riesgos / consideraciones	**Dependencia del proveedor** (APIs/límites), menor **control del SO/kernel**, posibles **restricciones** de personalización, **coste** al escalar.

3.1.3.D. IaaS (Infraestructure as a Service)

Aspecto	Resumen
Qué es	**Infraestructura como Servicio**: recursos **hardware remotos** (CPU, RAM, disco, red) para **ejecutar SO y apps** sin depender del equipo local.
Operación	**Pago por uso**. El proveedor gestiona **CPD, virtualización y redes**; tú creas **VMs/volúmenes/VPC** y **mantienes tu SO y software**.
Ejemplo	**AWS EC2**, **Azure VMs**, **Google Compute Engine** (y, encima, K8s: **EKS/AKS/GKE**).
Ventajas	**Escala a demanda**, aprovisionamiento rápido, **plantillas**, elección de **región/zona**, pagas lo que usas.
Riesgos	**Coste** por sobredimensionar/olvidar recursos; **parcheo y seguridad del SO invitado** son tuyos; diseño de **red/almacenamiento** impacta rendimiento; **dependencia de APIs**.

Veamos si ha quedado claro

- Un centro usa un CRM accesible por navegador; no instalan nada en local y pagan por usuario/mes.
- Un equipo sube su código a una plataforma que lo ejecuta y escala automáticamente, sin gestionar servidores
- Una empresa crea máquinas virtuales en la nube y les instala su propio sistema y aplicaciones.
- Un alumno instala LibreOffice en su portátil desde la tienda del SO y trabaja sin conexión.
- Un estudio usa una app web de edición de vídeo colaborativa; el render se hace en servidores del proveedor

Veamos si ha quedado claro

- Un dev despliega su API metida en un contenedor; la plataforma le da logs, routing y actualizaciones sin tocar el sistema.
- Un docente contrata almacenamiento y redes en la nube para montar su servidor Linux a medida.
- Un usuario descarga GIMP y lo instala en su PC para editar imágenes localmente.
- Un centro adopta correo y ofimática online con calendarios y drive integrados, todo vía web.

Veamos si ha quedado claro

- Un equipo usa base de datos “gestionada” de un proveedor; solo define tablas y datos, no administra sistema ni parches del motor.
- Una pyme ejecuta Windows Server en una VM de la nube y configura ahí su ERP.
- Un alumno compra y descarga un editor de audio de escritorio, con licencia por versión.

3.1.3.E. Tabla resumen

Modelo	Se ejecuta	Tipo de entrega	Qué pone/hace el proveedor	Qué pone/hace el usuario o cliente
SaaS	En la nube	Servicio (suscripción)	Aplicación completa operativa, infraestructura, parches, copias	Usuarios, configuración funcional, datos de negocio
PaaS	En la nube	Servicio (suscripción/uso)	Plataforma y runtimes, BD/servicios gestionados, escalado, logs	Tu código, variables/secretos, configuración de despliegue
IaaS	En la nube	Servicio (pago por uso)	Infraestructura: VMs, red, almacenamiento, hipervisor	SO invitado, parches del SO, firewall del SO, tu software y datos
SaaP	En local (por defecto)	Producto (licencia/versión)	Instalador/paquete y actualizaciones del fabricante	Instalación, configuración, parches locales, copias y seguridad propias